Optimistic vs Pessimistic Locking

Table of Contents

Learning Objectives

By the end of this section you will be able to:

- Explain the lost update problem and why it matters
- Choose between optimistic and pessimistic locking for a given workload
- Write correct `SELECT FOR UPDATE`, `SELECT FOR SHARE`, `NOWAIT`, and `SKIP LOCKED` queries
- Implement the version-column optimistic locking pattern from scratch
- Recognise deadlock risks introduced by pessimistic locking and mitigate them
- Describe the OCC (Optimistic Concurrency Control) algorithm step by step
- Articulate the trade-offs in a technical interview or system-design discussion

---

Theory

Lost Update Problem

The **lost update** is one of the classic concurrency anomalies. It occurs when two transactions read the same row, each modifies it based on the value read, and the second write silently overwrites the first write.

Scenario:

```
T1 reads account balance = 1000
T2 reads account balance = 1000
T1 writes balance = 1000 - 200 = 800   (debit $200)
T2 writes balance = 1000 + 500 = 1500  (credit $500)
Final balance = 1500  -- T1's debit is LOST
Correct balance should be = 1300
```

At **READ COMMITTED** isolation (PostgreSQL default), lost updates are possible if you use a read-modify-write pattern that does not lock the row. At **REPEATABLE READ** and **SERIALIZABLE**, PostgreSQL detects and

prevents lost updates. However, many applications run at READ COMMITTED for throughput reasons, so they must handle this explicitly.

Both pessimistic locking and optimistic locking are strategies to prevent the lost update without necessarily raising the isolation level for the entire connection.

Pessimistic Locking

Pessimistic locking assumes **conflict is likely**. Before reading a row you intend to modify, you acquire a lock so that no other transaction can modify it until you are done. The lock is held for the duration of the transaction.

SELECT FOR UPDATE syntax and behavior

```
SELECT ... FROM table WHERE ... FOR UPDATE;
```

- Acquires a **row-level exclusive lock** (`FOR UPDATE`) on every row returned by the query.
- Other transactions that attempt `SELECT FOR UPDATE` , `SELECT FOR SHARE` , or plain `UPDATE` / `DELETE` on those rows will **block** until the locking transaction commits or rolls back.
- Plain `SELECT` (without `FOR UPDATE`) is **never blocked** — MVCC lets readers see the snapshot.
- The lock is automatically released at `COMMIT` or `ROLLBACK` .
- Works well with `JOIN` : locks rows in all joined tables that are listed after `FOR UPDATE OF table_name` .

SELECT FOR SHARE

```
SELECT ... FROM table WHERE ... FOR SHARE;
```

- Acquires a **row-level share lock** (`FOR SHARE`).
- Multiple transactions can hold share locks on the same row simultaneously.
- Blocks `FOR UPDATE` and `UPDATE` / `DELETE` but allows concurrent `FOR SHARE` readers.
- Use case: a parent row must not be deleted while child rows are being inserted (referential integrity enforcement without a full exclusive lock).

NOWAIT and SKIP LOCKED options

NOWAIT — fail immediately instead of blocking:

```
SELECT ... FROM table WHERE ... FOR UPDATE NOWAIT;
```

If any row in the result set is already locked by another transaction, PostgreSQL raises:

```
ERROR:  could not obtain lock on row in relation "table"
```

The application catches this error and can retry, return an error to the user, or choose a different row.

SKIP LOCKED — skip rows that are currently locked:

```
SELECT ... FROM table WHERE ... FOR UPDATE SKIP LOCKED;
```

Instead of blocking or failing, PostgreSQL silently skips over rows held by other transactions and returns only unlocked rows. This is the canonical pattern for **multi-worker job queues**: each worker picks the next available (unlocked) job row without contending with other workers.

When to use pessimistic locking

- High-contention workloads where multiple processes compete for the same rows frequently.
- Long read-modify-write cycles where optimistic retry cost would be high.
- When the business consequence of a failed update is expensive (e.g., financial transfers).
- When you need to guarantee that no other writer touches a row while you are computing a new value.
- Inventory or seat-reservation systems with hard "only one winner" semantics.

Optimistic Locking

Optimistic locking assumes **conflict is rare**. No database lock is acquired at read time. Instead, the transaction records a "version token" when it reads a row and includes that token in the `WHERE` clause of the subsequent `UPDATE`. If the row was modified between the read and the write, the update affects zero rows, and the application detects the conflict and retries.

Version column pattern

Option A — Integer version counter:

```
ALTER TABLE accounts ADD COLUMN version INTEGER NOT NULL DEFAULT 0;
```

Option B — UUID/etag (useful when version must be opaque to clients):

```
ALTER TABLE accounts ADD COLUMN etag UUID NOT NULL DEFAULT gen_random_uuid();
```

Option C — Timestamp (less reliable due to clock precision):

```
ALTER TABLE accounts ADD COLUMN updated_at TIMESTAMPTZ NOT NULL DEFAULT now();
```

Integer version counters are the most common choice because they are easy to reason about and compare.

How it works: read version, update WHERE version = old_version, check rows affected

1. **Read phase**: `SELECT id, balance, version FROM accounts WHERE id = $1` — capture `version`.
2. **Compute phase**: calculate the new balance in application logic.
3. **Write phase**: `UPDATE accounts SET balance = $new, version = version + 1 WHERE id = $1 AND version = $old_version`.
4. **Check affected rows**: if `rowcount == 0`, a conflict occurred — another writer changed the row. Retry from step 1.
5. If `rowcount == 1`, the update succeeded.

No database lock is held between steps 1 and 3. The entire conflict detection happens atomically inside the single `UPDATE` statement because the `WHERE version = $old_version` predicate is evaluated under the statement's snapshot.

OCC algorithm

The **Optimistic Concurrency Control (OCC)** algorithm has three formal phases:

1. **Read phase** — transaction executes, reads data, performs all computations locally (no locks held).
2. **Validation phase** — before committing, the system checks whether any data read has been modified by a concurrent committed transaction.
3. **Write phase** — if validation passes, changes are written and committed. If validation fails, the transaction is aborted and restarted.

In the PostgreSQL version-column implementation:

- Read phase = the `SELECT`.
- Validation + Write phase = the single `UPDATE ... WHERE version = $old` which atomically validates and writes.

OCC is most efficient when the **probability of conflict is low**, because each conflict costs a full retry cycle.

When to use optimistic locking

- Low-to-medium contention: most transactions succeed on the first try.
- Read-heavy workloads with occasional writes.
- Distributed systems where taking a database lock is expensive or impossible.
- REST APIs following the ETag pattern (HTTP 412 Precondition Failed).
- When you cannot afford long-held locks (risk of deadlock or connection pool starvation).

Comparison

Dimension	Pessimistic Locking	Optimistic Locking
Lock acquisition	At read time (`SELECT FOR UPDATE`)	No lock at read time
Contention level	Best for high contention	Best for low/medium contention
Overhead (low load)	Higher (unnecessary blocking)	Lower (no lock round-trip)
Overhead (high load)	Predictable (waiters queue up)	High (many retries, wasted work)
Deadlock risk	Yes — must order lock acquisition	None (no locks held)
Retry logic	Not required	Required in application
Implementation	Simple SQL clause	Requires version column + retry loop
Throughput	Lower under contention (serialised)	Higher under low contention
Failure mode	Transaction waits (or NOWAIT error)	Transaction retries (or gives up)
Distributed systems	Difficult across nodes	Natural fit

ASCII Diagram

Optimistic Locking Flow with Two Concurrent Transactions

```
Time  Transaction A                       Transaction B
----  ----------------------------------  ----------------------------------
 t1   SELECT id, balance, version         SELECT id, balance, version
```

```
     FROM accounts WHERE id=1;            FROM accounts WHERE id=1;
     -- reads: balance=1000, version=5    -- reads: balance=1000, version=5

t2   [compute: new_balance = 800]         [compute: new_balance = 1500]

t3   UPDATE accounts                      (still computing...)
       SET balance=800,
           version=6
      WHERE id=1
        AND version=5;
     -- rowcount = 1  => SUCCESS
     COMMIT;
     -- Row now: balance=800, version=6

t4   (done)                               UPDATE accounts
                                            SET balance=1500,
                                                version=6
                                           WHERE id=1
                                             AND version=5;
                                          -- rowcount = 0  => CONFLICT!
                                          -- version is now 6, not 5
                                          ROLLBACK;

t5                                        [retry: re-read row]
                                          SELECT id, balance, version
                                          FROM accounts WHERE id=1;
                                          -- reads: balance=800, version=6

t6                                        [compute: new_balance = 800 + 500 = 1300]

t7                                        UPDATE accounts
                                            SET balance=1300,
                                                version=7
                                           WHERE id=1
                                             AND version=6;
                                          -- rowcount = 1  => SUCCESS
                                          COMMIT;
                                          -- Row now: balance=1300, version=7
```

Key insight: No locks were held between the SELECT and the UPDATE. Conflict detection was entirely via the version column predicate in the WHERE clause. The final result (1300) is correct.

SQL Examples

Example 1: Basic SELECT FOR UPDATE

```sql
-- Lock the account row before modifying it
BEGIN;

SELECT id, balance
  FROM accounts
```

```
 WHERE id = 42
   FOR UPDATE;

-- At this point no other transaction can UPDATE or DELETE account 42
-- Other transactions attempting FOR UPDATE on row 42 will BLOCK here

UPDATE accounts
   SET balance = balance - 100
 WHERE id = 42;

COMMIT;
```

Example 2: SELECT FOR UPDATE with JOIN

```
BEGIN;

-- Lock the order AND its customer row simultaneously
SELECT o.id, o.total, c.credit_limit
  FROM orders o
  JOIN customers c ON c.id = o.customer_id
 WHERE o.id = 999
   FOR UPDATE OF o           -- lock only the orders row
   FOR SHARE  OF c;          -- share-lock the customer row

-- Process the order...
UPDATE orders SET status = 'PROCESSING' WHERE id = 999;

COMMIT;
```

Example 3: SELECT FOR SHARE

```
BEGIN;

-- Allow concurrent readers but block writers
-- Used when we need the parent to stay while we insert children
SELECT id, name
  FROM departments
 WHERE id = 5
   FOR SHARE;

INSERT INTO employees (name, department_id)
VALUES ('Alice', 5);

COMMIT;
-- The department row could not be deleted while this transaction ran
```

Example 4: SKIP LOCKED — Multi-Worker Job Queue

```sql
-- Worker process: grab exactly one pending job, skip locked ones
BEGIN;

SELECT id, payload
  FROM job_queue
 WHERE status = 'PENDING'
 ORDER BY created_at
 LIMIT 1
   FOR UPDATE SKIP LOCKED;

-- If no row returned, queue is empty (or all jobs are being worked on)
-- Otherwise process the returned job

UPDATE job_queue
   SET status    = 'PROCESSING',
       started_at = now(),
       worker_id  = pg_backend_pid()
 WHERE id = :fetched_id;

COMMIT;
```

Example 5: NOWAIT — Fail Fast Instead of Blocking

```sql
BEGIN;

-- Try to lock the row; raise error immediately if locked
SELECT id, seat_number, status
  FROM seats
 WHERE id = 101
   FOR UPDATE NOWAIT;

-- If we reach here, we have the lock
UPDATE seats SET status = 'RESERVED', user_id = 7 WHERE id = 101;

COMMIT;

-- Caller should handle:
-- ERROR: could not obtain lock on row in relation "seats"
```

Example 6: Optimistic Locking — Table Setup

```sql
CREATE TABLE products (
    id         SERIAL PRIMARY KEY,
    name       TEXT          NOT NULL,
    stock      INTEGER       NOT NULL CHECK (stock >= 0),
    price      NUMERIC(10,2) NOT NULL,
    version    INTEGER       NOT NULL DEFAULT 0,
    updated_at TIMESTAMPTZ   NOT NULL DEFAULT now()
);
```

```
INSERT INTO products (name, stock, price)
VALUES ('Widget A', 100, 9.99);
```

Example 7: Optimistic Locking — Full Read-Modify-Write Cycle

```
-- Step 1: Read (no lock)
SELECT id, stock, version
  FROM products
 WHERE id = 1;
-- Returns: id=1, stock=100, version=0

-- Step 2: Application computes new stock = 100 - 5 = 95

-- Step 3: Conditional update
UPDATE products
   SET stock      = 95,
       version    = version + 1,
       updated_at = now()
 WHERE id      = 1
   AND version = 0;   -- <-- the optimistic check

-- GET DIAGNOSTICS affected_rows = ROW_COUNT;
-- IF affected_rows = 0 THEN RAISE EXCEPTION 'conflict'; END IF;
```

Example 8: Optimistic Locking with UUID etag

```
CREATE TABLE documents (
    id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title   TEXT    NOT NULL,
    body    TEXT    NOT NULL,
    etag    UUID    NOT NULL DEFAULT gen_random_uuid()
);

-- Read (client stores the etag, e.g. in HTTP ETag header)
SELECT id, title, body, etag FROM documents WHERE id = $1;

-- Update (client sends back the etag it received)
UPDATE documents
   SET title = $new_title,
       body  = $new_body,
       etag  = gen_random_uuid()   -- rotate the etag on every write
 WHERE id   = $doc_id
   AND etag = $client_etag;        -- If-Match check

-- 0 rows affected = HTTP 412 Precondition Failed
```

Example 9: CAS (Compare-And-Swap) Pattern

```
-- Atomically set status to 'ACTIVE' only if currently 'PENDING'
UPDATE workflow_tasks
   SET status     = 'ACTIVE',
       started_at = now()
 WHERE id     = $1
   AND status = 'PENDING';   -- CAS predicate

-- Returns 0 rows if task was already grabbed by another worker
-- Returns 1 row if this worker won the race
```

Example 10: Retry Loop in PL/pgSQL

```
CREATE OR REPLACE FUNCTION deduct_stock(
    p_product_id INTEGER,
    p_quantity    INTEGER,
    p_max_retries INTEGER DEFAULT 5
)
RETURNS VOID LANGUAGE plpgsql AS $$
DECLARE
    v_version INTEGER;
    v_stock   INTEGER;
    v_rows    INTEGER;
    v_attempt INTEGER := 0;
BEGIN
    LOOP
        v_attempt := v_attempt + 1;
        IF v_attempt > p_max_retries THEN
            RAISE EXCEPTION 'optimistic lock: max retries exceeded for product %',
                            p_product_id;
        END IF;

        -- Read phase
        SELECT stock, version
          INTO v_stock, v_version
          FROM products
         WHERE id = p_product_id;

        IF v_stock < p_quantity THEN
            RAISE EXCEPTION 'insufficient stock';
        END IF;

        -- Write phase (validate + write atomically)
        UPDATE products
           SET stock   = v_stock - p_quantity,
               version = v_version + 1
         WHERE id      = p_product_id
           AND version = v_version;

        GET DIAGNOSTICS v_rows = ROW_COUNT;
```

```
        EXIT WHEN v_rows = 1;   -- success

        -- v_rows = 0 means conflict; loop and retry
        PERFORM pg_sleep(0.01 * v_attempt);  -- simple back-off
    END LOOP;
END;
$$;
```

Example 11: Advisory Locks as Lightweight Pessimistic Locks

```
-- Session-level advisory lock on an integer key (e.g., user ID)
SELECT pg_advisory_lock(42);

-- ... do work ...

SELECT pg_advisory_unlock(42);

-- Transaction-level advisory lock (auto-released on COMMIT/ROLLBACK)
BEGIN;
SELECT pg_advisory_xact_lock(42);
-- ... work ...
COMMIT;

-- Try-lock variant (returns boolean, never blocks)
SELECT pg_try_advisory_lock(42);
```

Example 12: Combining SKIP LOCKED with a Status Column

```
-- Reliable at-least-once job queue
WITH next_job AS (
    SELECT id
      FROM jobs
     WHERE status = 'queued'
     ORDER BY priority DESC, created_at ASC
     LIMIT 1
       FOR UPDATE SKIP LOCKED
)
UPDATE jobs
   SET status      = 'running',
       locked_by   = inet_client_addr(),
       locked_at   = now()
  FROM next_job
 WHERE jobs.id = next_job.id
RETURNING jobs.*;
```

Example 13: SELECT FOR UPDATE with LIMIT (Anti-pattern Warning)

```
-- WARNING: FOR UPDATE with LIMIT does NOT guarantee the same rows
-- that LIMIT chose are the ones locked (pre-9.3 issue, now fixed,
-- but still document ordering explicitly)

-- CORRECT pattern: ensure deterministic ordering
SELECT id, balance
  FROM accounts
 WHERE status = 'active'
 ORDER BY id          -- deterministic order avoids accidental deadlocks
 LIMIT 10
   FOR UPDATE;
```

Example 14: Checking Lock Contention

```
-- See who is waiting for row locks right now
SELECT
    blocked.pid                         AS blocked_pid,
    blocked.query                       AS blocked_query,
    blocking.pid                        AS blocking_pid,
    blocking.query                      AS blocking_query,
    now() - blocked.query_start         AS wait_duration
FROM  pg_stat_activity blocked
JOIN  pg_stat_activity blocking
      ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
WHERE blocked.wait_event_type = 'Lock';
```

Example 15: Optimistic Locking in a Transaction (Multi-row)

```
BEGIN;

-- Read both rows, capture versions
SELECT id, balance, version FROM accounts WHERE id IN (1, 2) ORDER BY id;
-- Returns: (1, 500, 3), (2, 200, 7)

-- Transfer $100 from account 1 to account 2
UPDATE accounts SET balance = balance - 100, version = version + 1
 WHERE id = 1 AND version = 3;
-- Check rowcount

UPDATE accounts SET balance = balance + 100, version = version + 1
 WHERE id = 2 AND version = 7;
-- Check rowcount

-- If either rowcount = 0, ROLLBACK and retry
COMMIT;
```

Example 16: FOR UPDATE OF in Multi-Table Query

```sql
BEGIN;

SELECT p.id, p.stock, s.name AS supplier_name
  FROM products p
  JOIN suppliers s ON s.id = p.supplier_id
 WHERE p.category = 'electronics'
   AND p.stock < 10
   FOR UPDATE OF p;   -- lock only products rows, not suppliers

-- Replenish stock
UPDATE products SET stock = stock + 50, version = version + 1
 WHERE id = ANY(ARRAY(SELECT p.id FROM products p
                      WHERE p.category = 'electronics'
                        AND p.stock < 10));

COMMIT;
```

Common Mistakes

1. **Using optimistic locking with a timestamp version column on a busy system.** If two transactions execute within the same millisecond (or microsecond on modern hardware), they both read the same `updated_at` value, both update successfully, and you still have a lost update. Always use an integer counter or UUID etag — never a timestamp alone.

2. **Forgetting to check `ROW_COUNT` after the optimistic UPDATE.** The update silently succeeds with 0 rows affected if there is a version mismatch. If the application does not check the affected row count and retry, it believes the write succeeded when it did not. This is a silent data-correctness bug.

3. **Holding a `SELECT FOR UPDATE` lock while making external HTTP calls.** The lock is held for the entire duration of the transaction. An external call that takes 2–30 seconds will block all other writers on those rows for that entire time, causing connection pool exhaustion and cascading failures.

4. **Not ordering `FOR UPDATE` rows consistently across transactions.** Transaction A locks row 1 then row 2; Transaction B locks row 2 then row 1. This classic cycle produces a deadlock. Always sort the rows by primary key (or any consistent ordering) before locking them.

5. **Using `SKIP LOCKED` for work that requires exactly-once semantics.** `SKIP LOCKED` guarantees at-most-once delivery per lock acquisition, but if the worker crashes without updating the status, the job stays `PENDING` forever. You need a heartbeat or timeout-based requeue mechanism as well.

6. **Applying optimistic locking only to some update paths.** If any code path updates a row without checking the version, the version column provides no protection through that path. All writers must participate.

7. **Infinite retry loops without a back-off or max-retry cap.** Under high contention, a naive `LOOP … RETRY` can spin-lock the CPU. Add exponential back-off with jitter and a maximum retry count.

Best Practices

1. **Choose the strategy based on your conflict rate.** Measure or estimate how often two transactions will attempt to write the same row concurrently. Below ~5% conflict rate, optimistic locking is usually more efficient. Above ~20%, pessimistic locking serialises work more predictably.
2. **Always include a deterministic ORDER BY when using `SELECT FOR UPDATE` with `LIMIT`.** This prevents accidental deadlocks caused by different transactions locking the same set of rows in different orders.
3. **Keep pessimistically-locked transactions as short as possible.** Every millisecond a lock is held is a millisecond another transaction is blocked. Do all external I/O and computation before `BEGIN`, not inside it.
4. **Wrap optimistic-locking retries in exponential back-off with jitter.** `pg_sleep(random() * 0.1 * 2^attempt)` prevents a thundering-herd effect where all retrying clients hit the database at the same moment.
5. **Encapsulate the retry loop in a helper function or middleware layer.** Business logic should not be polluted with locking infrastructure. A single `with_optimistic_retry(fn, max_retries=5)` wrapper keeps the codebase clean.
6. **Test your optimistic locking under synthetic contention.** Use pgbench or concurrent psql sessions to verify that your retry logic actually handles `rowcount = 0` correctly before shipping to production.
7. **Document which columns are "version columns" in your schema.** Future developers will be confused if a `version INTEGER` column appears without explanation. Add a comment: `COMMENT ON COLUMN accounts.version IS 'Optimistic lock version; increment on every UPDATE';`

Performance Considerations

- **Lock overhead is real even with no contention.** `SELECT FOR UPDATE` acquires a lock manager entry, adds to the row's `xmax`, and must update the lock table. Under no-contention conditions, this adds ~0.1–0.5 ms per row.
- **Optimistic locking has zero overhead when there is no conflict.** The version check is just an extra predicate in the WHERE clause — essentially free.
- **SKIP LOCKED is very efficient for queues.** The planner can stop scanning as soon as it has collected `LIMIT` unlocked rows. It does not scan the entire table.
- **Retry cost under high contention can exceed pessimistic cost.** If 10 transactions all try to update the same row optimistically, 9 of them will retry. Each retry involves a new read + a failed write. Under high contention, pessimistic locking with a queue (FIFO waiters) is actually more efficient because no work is wasted.
- **Index your version column if you query by it.** Rare, but if your application does `WHERE id = $1 AND version = $2` and the optimizer chooses to use the version column in a multi-column index, ensure that index exists.

- **Connection pool sizing interacts with pessimistic locking.** A pool of N connections can hold at most N locks simultaneously. If N is small and transactions hold locks for a long time, the pool becomes the bottleneck, not the database.

Interview Questions and Answers

Q1: What is the lost update problem and which PostgreSQL isolation levels prevent it?

A: The lost update problem occurs when two transactions each read a value, compute a new value based on the read, and write it back — the second write overwrites the first write's changes. PostgreSQL's READ COMMITTED isolation does not prevent lost updates if the application uses a read-modify-write pattern without explicit locking. REPEATABLE READ and SERIALIZABLE both prevent lost updates: at REPEATABLE READ, the second UPDATE will either block until the first commits (and then see the new value) or raise a serialization failure. You can also prevent lost updates at READ COMMITTED by using SELECT FOR UPDATE or by relying on a version-column optimistic lock.

Q2: What is the difference between SELECT FOR UPDATE and SELECT FOR SHARE?

A: `SELECT FOR UPDATE` acquires an exclusive row lock that blocks any other transaction from acquiring FOR UPDATE, FOR SHARE, UPDATE, or DELETE on those rows. `SELECT FOR SHARE` acquires a shared row lock that blocks writers (UPDATE/DELETE and FOR UPDATE) but allows other readers to also hold FOR SHARE locks on the same rows simultaneously. FOR SHARE is appropriate when you need a parent row to remain stable while you perform work referencing it (e.g., inserting child rows), but you do not need to prevent other readers from also referencing that parent.

Q3: Explain SKIP LOCKED and give a real-world use case.

A: SKIP LOCKED is an option for `SELECT FOR UPDATE/SHARE` that makes the query silently skip any rows that are currently locked by another transaction instead of blocking or raising an error. The canonical use case is a multi-worker job queue: each worker runs `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1` to grab one unlocked pending job. Because each worker skips rows already held by other workers, there is no lock contention at all — every worker immediately gets a distinct job without waiting.

Q4: How does optimistic locking prevent the lost update without holding any database locks?

A: The version column acts as a conditional write gate. When transaction T reads a row, it captures the current version number. When T writes the row, it includes `WHERE version = <old_version>` in the UPDATE. Because the UPDATE is atomic, if another transaction has already incremented the version (by writing the row), the predicate `version = <old_version>` no longer matches, and the UPDATE affects zero rows. The application detects the zero row count and knows a conflict occurred, so it re-reads and retries. At no point does any transaction hold a lock between the read and the write.

Q5: When would you prefer pessimistic locking over optimistic locking?

A: Pessimistic locking is preferable when: (a) conflict probability is high (many concurrent writers on the same rows), making optimistic retries expensive; (b) the compute cost of re-executing the transaction is high (e.g., involves complex aggregations or external API calls); (c) the business logic must guarantee that once a row is read it cannot be changed before the write (seat reservations, inventory decrement where the user sees a specific price/availability during the transaction); (d) you cannot afford to show users stale data during a retry cycle.

Q6: What does NOWAIT do and when should you use it?

A: `NOWAIT` modifies `SELECT FOR UPDATE/SHARE` to raise an error immediately (`ERROR: could not obtain lock on row`) if any row in the result is already locked, instead of waiting for the lock to become available. Use NOWAIT when: (a) you want to give the user immediate feedback ("this resource is busy, try again") rather than making them wait for an indeterminate time; (b) you have a hard latency SLA and cannot afford to queue; (c) you want to implement optimistic-style "try once and fail fast" semantics while still using a lock-based approach.

Q7: How do you prevent deadlocks when using pessimistic locking across multiple rows?

A: Deadlocks arise when transaction A holds lock on row 1 and waits for row 2, while transaction B holds lock on row 2 and waits for row 1. The solution is to always acquire locks in a consistent, deterministic order across all transactions — typically by sorting rows by primary key before locking them (`ORDER BY id FOR UPDATE`). Additional mitigations: keep transactions short, avoid acquiring locks on rows you may not need, and consider using NOWAIT to break potential deadlocks at the application layer.

Q8: Can you use optimistic locking across multiple rows in a single transaction?

A: Yes, but each row must have its own version column and each UPDATE must include its own version check. If any UPDATE in the transaction returns zero rows, the entire transaction should be rolled back and retried from the beginning. This multi-row optimistic lock provides correctness equivalent to REPEATABLE READ semantics for the specific rows involved, without holding any locks. The retry cost grows with the number of rows because any single row conflicting forces a full retry.

Exercises with Solutions

Exercise 1: Implement a ticket reservation system using pessimistic locking

Task: Write a `reserve_seat(p_event_id, p_seat_id, p_user_id)` function that locks the seat row, checks it is available, and marks it reserved. Handle the case where the seat is already taken.

Solution:

```
CREATE OR REPLACE FUNCTION reserve_seat(
    p_event_id INTEGER,
    p_seat_id  INTEGER,
    p_user_id  INTEGER
)
RETURNS TEXT LANGUAGE plpgsql AS $$
DECLARE
    v_status TEXT;
BEGIN
    -- Pessimistic lock: block until we can exclusively check this seat
    SELECT status
      INTO v_status
      FROM seats
     WHERE event_id = p_event_id
       AND seat_id  = p_seat_id
       FOR UPDATE;

    IF NOT FOUND THEN
        RETURN 'ERROR: seat does not exist';
    END IF;
```

```
    IF v_status <> 'available' THEN
        RETURN 'CONFLICT: seat already ' || v_status;
    END IF;

    UPDATE seats
       SET status    = 'reserved',
           user_id   = p_user_id,
           reserved_at = now()
     WHERE event_id = p_event_id
       AND seat_id  = p_seat_id;

    RETURN 'SUCCESS';
END;
$$;
```

Exercise 2: Implement optimistic locking with retry for a bank transfer

Task: Write a `transfer(from_id, to_id, amount)` stored procedure using optimistic locking with up to 3 retries.

Solution:

```
CREATE OR REPLACE PROCEDURE transfer(
    p_from   INTEGER,
    p_to     INTEGER,
    p_amount NUMERIC
)
LANGUAGE plpgsql AS $$
DECLARE
    v_from_balance  NUMERIC;
    v_from_version  INTEGER;
    v_to_version    INTEGER;
    v_rows          INTEGER;
    v_attempt       INTEGER := 0;
BEGIN
    LOOP
        v_attempt := v_attempt + 1;
        IF v_attempt > 3 THEN
            RAISE EXCEPTION 'transfer failed after 3 attempts (contention)';
        END IF;

        SELECT balance, version INTO v_from_balance, v_from_version
          FROM accounts WHERE id = p_from;
        SELECT version INTO v_to_version
          FROM accounts WHERE id = p_to;

        IF v_from_balance < p_amount THEN
            RAISE EXCEPTION 'insufficient funds';
        END IF;

        UPDATE accounts SET balance = balance - p_amount,
```

```
                        version = version + 1
         WHERE id = p_from AND version = v_from_version;
        GET DIAGNOSTICS v_rows = ROW_COUNT;
        IF v_rows = 0 THEN CONTINUE; END IF;

        UPDATE accounts SET balance = balance + p_amount,
                        version = version + 1
         WHERE id = p_to AND version = v_to_version;
        GET DIAGNOSTICS v_rows = ROW_COUNT;
        IF v_rows = 0 THEN
            -- Undo the first update by rolling back
            ROLLBACK;
            CONTINUE;
        END IF;

        EXIT;
    END LOOP;
END;
$$;
```

Exercise 3: SKIP LOCKED job queue

Task: Write a query that atomically claims the next 3 highest-priority pending jobs for a worker, skipping any currently claimed.

Solution:

```
WITH claimed AS (
    SELECT id
      FROM jobs
     WHERE status = 'pending'
     ORDER BY priority DESC, created_at ASC
     LIMIT 3
       FOR UPDATE SKIP LOCKED
)
UPDATE jobs
   SET status     = 'running',
       claimed_at = now(),
       worker_id  = $1
  FROM claimed
 WHERE jobs.id = claimed.id
RETURNING jobs.id, jobs.payload;
```

Cross-references

- `05_deadlocks.md` — deadlock detection and prevention; ordering locks to avoid cycles
- `07_serializable_snapshot_isolation.md` — SSI as an alternative to explicit locking for correctness
- `04_isolation_levels.md` — how REPEATABLE READ and SERIALIZABLE prevent lost updates automatically

- `08_transaction_interview_guide.md` — Q33–Q35 cover SKIP LOCKED and NOWAIT interview questions
- `../02_MVCC/` — understanding MVCC explains why plain SELECT never blocks under any locking strategy
- `../10_PostgreSQL_Internals/` — lock manager internals, pg_locks system catalog